How Robust Is Character-Level Transliteration to Informal Typing? A Paired Noise Benchmark for Romanized Tamil

Lebi Raja
Department of AI & Data Science
Karunya Institute of Technology and Sciences, India

***Abstract*—Tamil speakers routinely type Tamil in Latin letters ("Tanglish") and software must convert it back to Tamil script. Published transliteration models are trained and tested on carefully romanized text, so their behaviour on hurried, informal typing is largely unmeasured. We study this gap for word-level Latin-to-Tamil transliteration with character-level sequence-to-sequence models. We contribute (i) a paired robustness benchmark that applies seven Tamil-specific noise operators at four severities to the Dakshina test lexicon, so every noisy input is tied to its clean form and gold answer, together with a real-chat word set and a team-typed set; (ii) an augmentation study comparing random character noise, Tamil-phonological noise and sampled back-transliteration against a more-data control; and (iii) a parameter-matched comparison of a BiLSTM with attention and a small pre-LN Transformer, with five seeds and equal tuning budget. Our implementation reproduces the published Dakshina baselines within one CER point. Results of the benchmark and the augmentation study are reported in Section VI.**

***Index Terms*—transliteration, Tamil, robustness, data augmentation, sequence-to-sequence, romanization**

I. Introduction

Most Tamil speakers type on phones using the Latin alphabet. Search, autocomplete and display all need the text converted back into Tamil script, which is a transliteration task: sounds are mapped to letters, not meanings to words. The Latin alphabet cannot express several Tamil distinctions, including vowel length, doubled consonants and the several n, l and r sounds, and writers spell the same word in different ways. Informal typing adds more variation: dropped vowels (*kankl* for *kanakkil*), collapsed long vowels, collapsed doubled consonants, swapped digraphs and stretched letters.

Neural transliteration is well studied on clean data. The Dakshina dataset provides romanizations of Tamil words written by native speakers on request [1], and strong systems have been trained on larger corpora [2]. However, the Tamil text in Dakshina is encyclopaedic and was typed carefully, so it does not contain the shorthand found in chat. We therefore ask how much character-level models lose under informal typing, which training strategy recovers it, and whether the choice of architecture matters.

The literature also disagrees on architecture. On Dakshina Tamil, an LSTM beat a Transformer in one study [1], whereas a Transformer was better in another [3], and batch size can reverse the ranking [4]. None of these comparisons tested noisy input.

Contributions. (1) A paired, seeded robustness benchmark for Tamil with seven noise operators and four severities. (2) A study of four augmentation strategies and a more-data control, evaluated on held-out noise types and on real typing. (3) A parameter-matched, equal-tuning comparison of BiLSTM-attention and Transformer over five seeds with paired significance tests. (4) A reproducibility package: code, configurations, per-run logs and data checksums.

II. Related Work

Transliteration datasets and systems. Dakshina covers twelve South Asian languages with native-script Wikipedia text, a romanization lexicon and romanized sentences [1]. Aksharantar and the IndicXlit model extend coverage to 21 languages with millions of pairs [2]. Earlier systems include Brahmi-Net [5] and the shared tasks collected in NEWS [6]. Context-aware and large-model approaches have recently been compared with character models [3], [7].

Noisy and informal romanization. Keyboard data show how users romanize at scale [8], and a large study documents Latin-script variation across South Asian languages [9]. For Tamil, a shorthand-aware study examines vowel dropping but evaluates no neural model [10], and a Malayalam study reports CER rising from 7.4 to 22.7 when vowels are dropped [11]. Code-mixed Tamil corpora provide real informal text without Tamil-script labels [12].

Robustness and augmentation. Character models are brittle to noise unless trained with it [13], and mixing noise types helps [14]. Synthetic variation improves romanized language identification [15], and sampled back-translation is a standard way to create synthetic parallel data [16]. We transfer these ideas to Tamil transliteration.

Evaluation practice. Parameter confounds and weak statistics can reverse conclusions in character-level transduction [17]; we match model size and tuning budget and report seed variance.

III. Data

Training and clean evaluation. We use the Dakshina Tamil romanization lexicon: 25,000 training words (68,218 pairs), 2,500 development words (6,827 pairs) and 2,500 test words (6,864 pairs) [1]. Splits are word-disjoint. Inputs are lowercased Latin; outputs are NFC-normalized Tamil code

TABLE I
NOISE OPERATORS. p IS THE PER-POSITION PROBABILITY (REPEAT: PER WORD; KEY: SCALED BY 0.25).

Operator	Effect
V-DROP	delete non-initial vowels
V-LEN	collapse doubled vowels
GEM	collapse doubled consonants
DIGRAPH	th/t, dh/d, sh/s, ch/c/s swaps; zh→l/z
KEY	adjacent QWERTY key substitution
REPEAT	stretch one vowel by 1–3 letters
MIX	random combination of the above

points. A romanization with several attested native forms is scored against all of them.

Independent test. Aksharantar's Tamil test file contains every Dakshina test pair. We keep only rows whose word occurs in no Dakshina split, giving three subsets: AK-Freq (2,607 inputs), AK-NEI (991) and AK-NEF (706).

Synthetic noisy test sets. Seven operators act on the romanized input only (Table I). Each of 6,833 distinct test inputs is perturbed at severity $p \in \{0.1, 0.2, 0.3, 0.5\}$ with a seed derived from the word, operator and severity, so sets are exactly reproducible and each noisy item stays paired with its clean form and references. We report the realised character-edit rate of every set rather than the nominal p.

Real-typing sets. A word set drawn from the authors' own private chat exports (words only, no messages) was labelled with Tamil spellings by a language model and is being verified by the authors; and a team-typed set of about 1,500 Dakshina test words is collected under a fixed protocol. *Both sets are in preparation; their sizes and verification status are reported in Section VI once final.*

Synthetic training data. For the augmentation study (Section IV) we build 60,000 additional Tamil words from the IndicXlit unigram list, removing every Dakshina word, every independent-test word and any word sharing a five-code-point stem prefix with a held-out word. A Tamil-to-Latin BiLSTM trained on Dakshina romanizes them, giving 123,488 quality-filtered sampled pairs and a 60,000-pair greedy control.

IV. METHOD

Models. Both models are character-level with separate source and target vocabularies built from training data. The BiLSTM uses a two-layer bidirectional encoder, a two-layer LSTM decoder and Luong general attention [18], [19]. The Transformer is a pre-LN encoder-decoder with three layers per side and learned positions [20]. Each architecture is built at about 1M, 3M and 8M trainable parameters by choosing the width.

Training and decoding. Adam, cross-entropy with optional label smoothing, gradient clipping at 1.0, early stopping on development CER. Decoding uses beam search with beam 10 to produce an n-best list.

Metrics. CER is the corpus-level Levenshtein distance over code points between the top hypothesis and the closest attested reference, divided by reference length. Top-k accuracy counts an input correct if any of the best k hypotheses equals any reference; MRR uses the rank of the first correct hypothesis.

Augmentation conditions. A0: Dakshina only. A1: random character noise (deletion, insertion, substitution, swap). A2: the Tamil-phonological operators. A3: Dakshina plus sampled back-transliteration. A4: A2 plus A3. CTRL: Dakshina plus the same pool words romanized greedily, which separates more data from more spelling variation. Online noise is re-drawn every epoch with 40% of items left clean; held-out variants of A2 omit one operator and are tested on it.

Statistics. Five seeds per configuration; mean and standard deviation; paired bootstrap over test items for system pairs; Kruskal–Wallis with Dunn post-hoc for more than two systems; Spearman correlation between synthetic and human-set degradation.

V. EXPERIMENTAL SETUP

All models are trained with PyTorch 2.11 (CUDA 12.8) on an RTX 5070 (12 GB). Hyperparameters are chosen by 20 random-search trials per architecture and size using development CER only; test data are never used for selection.

Reproduction check. Before any new experiment we verify that our implementation matches published results. With one seed and 3M parameters, our BiLSTM reaches development CER 8.95 and top-1 62.8% against 8.4 and 65.3% reported by Roark et al. [1], and our Transformer reaches CER 9.38 against 9.4. Both CERs are within one point of the published values; the 2.5-point top-1 gap for the BiLSTM may come from differences in how references are counted and is not hidden.

VI. RESULTS

Full results (C3 architecture comparison, C1 degradation curves, C2 augmentation, real-typing sets) are produced by the experiment queue on the lab server and will be inserted here from the logged JSON files. The table and figure templates are fixed in the project plan; no number is reported until it comes from a logged run.

VII. LIMITATIONS

Dakshina is formal text romanized on request, not chat; we therefore speak of informal *romanized* Tamil and test on separate real-typing sets. Synthetic noise is a model of typing, and its realism is tested rather than assumed. Back-transliterated training pairs inherit the errors of the model that made them. The real-chat set is small, labelled by a language model and verified by the authors, and comes from a few private conversations. The team-typed set has few typists and was typed from Tamil-script prompts, which may be more careful than spontaneous chat. Vowel-dropped inputs can be genuinely ambiguous, so we report top-k accuracy as well as top-1.

References

[1] B. Roark et al., "Processing South Asian languages written in the Latin script: The Dakshina dataset," in *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC 2020)*, Marseille, France: European Language Resources Association, 2020, pp. 2413–2423.

[2] Y. Madhani et al., "Aksharantar: Open Indic-language transliteration datasets and models for the next billion users," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, Read as arXiv:2205.03018; venue and pages as cited in Baiju et al. 2024 and Azam et al. 2025, Singapore, 2023, pp. 40–57.

[3] C. Kirov, C. Johny, A. Katanova, A. Gutkin, and B. Roark, "Context-aware transliteration of romanized South Asian languages," *Computational Linguistics*, vol. 50, no. 2, pp. 475–534, 2024. DOI: 10.1162/coli_a_00510

[4] S. Wu, R. Cotterell, and M. Hulden, *Applying the transformer to character-level transduction*, Published at EACL 2021 (venue not printed in the arXiv PDF read), 2021. arXiv: 2005.10213 [cs.CL].

[5] A. Kunchukuttan, R. Puduppully, and P. Bhattacharyya, "Brahmi-Net: A transliteration and script conversion system for languages of the Indian subcontinent," in *Proceedings of NAACL-HLT 2015: Demonstrations*, Denver, Colorado, 2015, pp. 81–85.

[6] N. Chen, R. E. Banchs, M. Zhang, X. Duan, and H. Li, "Report of NEWS 2018 named entity transliteration shared task," in *Proceedings of the Seventh Named Entities Workshop*, Melbourne, Australia, 2018, pp. 55–73.

[7] G. Azam et al., *Beyond specialization: Benchmarking LLMs for transliteration of Indian languages*, 2025. arXiv: 2505.19851 [cs.CL].

[8] L. Hellsten et al., "Transliterated mobile keyboard input via weighted finite-state transducers," in *Proceedings of the 13th International Conference on Finite State Methods and Natural Language Processing (FSMNLP 2017)*, Umeå, Sweden, 2017, pp. 10–19. DOI: 10.18653/v1/W17-4002

[9] L. Wolf-Sonkin, V. Schogol, B. Roark, and M. Riley, "Latin script keyboards for South Asian languages with finite-state normalization," in *Proceedings of the 14th International Conference on Finite-State Methods and Natural Language Processing*, Dresden, Germany, 2019, pp. 108–117.

[10] H. M. A. D. Herath and T. G. D. K. Sumanathilaka, "TAMZHI: Shorthand romanized Tamil to Tamil reverse transliteration using novel hybrid approach," *International Journal on Advances in ICT for Emerging Regions (ICTer)*, vol. 17, no. 1, pp. 1–7, 2024. DOI: 10.4038/icter.v17i1.7271

[11] B. Baiju, K. Manohar, L. G. Pillai, and E. Sherly, *Romanized to native Malayalam script transliteration using an encoder-decoder framework*, IndoNLP 2025 shared-task system (team NexText), 2024. arXiv: 2412.09957 [cs.CL].

[12] B. R. Chakravarthi, V. Muralidaran, R. Priyadharshini, and J. P. McCrae, *Corpus creation for sentiment analysis in code-mixed Tamil-English text*, Published at SLTU–CCURL 2020 per arXiv listing (venue not printed in the PDF read), 2020. arXiv: 2006.00206 [cs.CL].

[13] Y. Belinkov and Y. Bisk, "Synthetic and natural noise both break neural machine translation," in *International Conference on Learning Representations (ICLR)*, 2018.

[14] V. Karpukhin, O. Levy, J. Eisenstein, and M. Ghazvininejad, *Training on synthetic noise improves robustness to natural noise in machine translation*, Published at W-NUT 2019 per arXiv listing (venue not printed in the PDF read), 2019. arXiv: 1902.01509 [cs.CL].

[15] A. Benton, A. Gutkin, C. Kirov, and B. Roark, *Improving informally romanized language identification*, 2025. arXiv: 2504.21540 [cs.CL].

[16] R. Sennrich, B. Haddow, and A. Birch, *Improving neural machine translation models with monolingual data*, Published at ACL 2016 (venue not printed in the arXiv PDF read), 2016. arXiv: 1511.06709 [cs.CL].

[17] M. Moran and C. Lignos, "Effective architectures for low resource multilingual named entity transliteration," in *Proceedings of the 3rd Workshop on Technologies for MT of Low Resource Languages*, 2020, pp. 79–86.

[18] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Read as arXiv:1508.04025v5; pages not verified, 2015.

[19] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations (ICLR)*, 2015.

[20] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, Read as arXiv:1706.03762v7; pages as cited in Roark et al. 2020, 2017, pp. 5998–6008.